

What We Proposed

● Data & Ingestion

- Ingest games + ratings from IGDB
- Ingest ratings from Steam
- Ingest critic scores from OpenCritic
- Normalize all sources into one Cloud SQL database
- Keep the catalog current automatically

● Search & Ranking

- Blend critic and user scores into one ranking
- User-configurable score weighting
- Search the full catalog by title
- Filter by genre, platform, developer, publisher, year
- Filter by minimum rating counts

● Cloud Deployment

- Web service as a Docker container on Cloud Run
- Ingestion scripts as a second container
- Cloud Run Job on a Cloud Scheduler trigger
- All database access inside the cloud, not from a local client

● Stretch Goals

User accounts via Identity Platform · Personal ratings and libraries · ML recommendations drawn from per-game metadata (genres, themes, keywords)

What We Built

Delivered

- IGDB + Steam ingestion: ~305k games, ~5.4M rows
- Blended critic/user ranking with adjustable weighting
- Title search across the full catalog
- Genre include/exclude, year range, score and count floors
- Web service + ingestion job, both on Cloud Run
- Scheduled automatic refresh via Cloud Scheduler

Changed Course

- Metacritic replaced OpenCritic as the third source - it arrives free with the Steam details call
- Platform / developer / publisher: data is ingested and queryable, but no web controls yet

Cut

- User accounts via Identity Platform
- Personal ratings and libraries
- ML-based recommendations

Everything cut was a stretch goal. Every core feature from the proposal shipped, with one source substitution.

Platform: Google Cloud



Cloud Run Service

Hosts the containerized Flask web app. Scales to zero, public HTTPS endpoint, no VM to manage.



Cloud Run Jobs

Runs the ingestion container to completion and exits - a batch workload, not a server.



Cloud SQL

Managed PostgreSQL 18. The system of record for every game, score and attribute.



Cloud Scheduler

Cron trigger that fires the ingestion job automatically, no machine left running.



Artifact Registry

Stores the two versioned Docker images the service and job deploy from.



Secret Manager

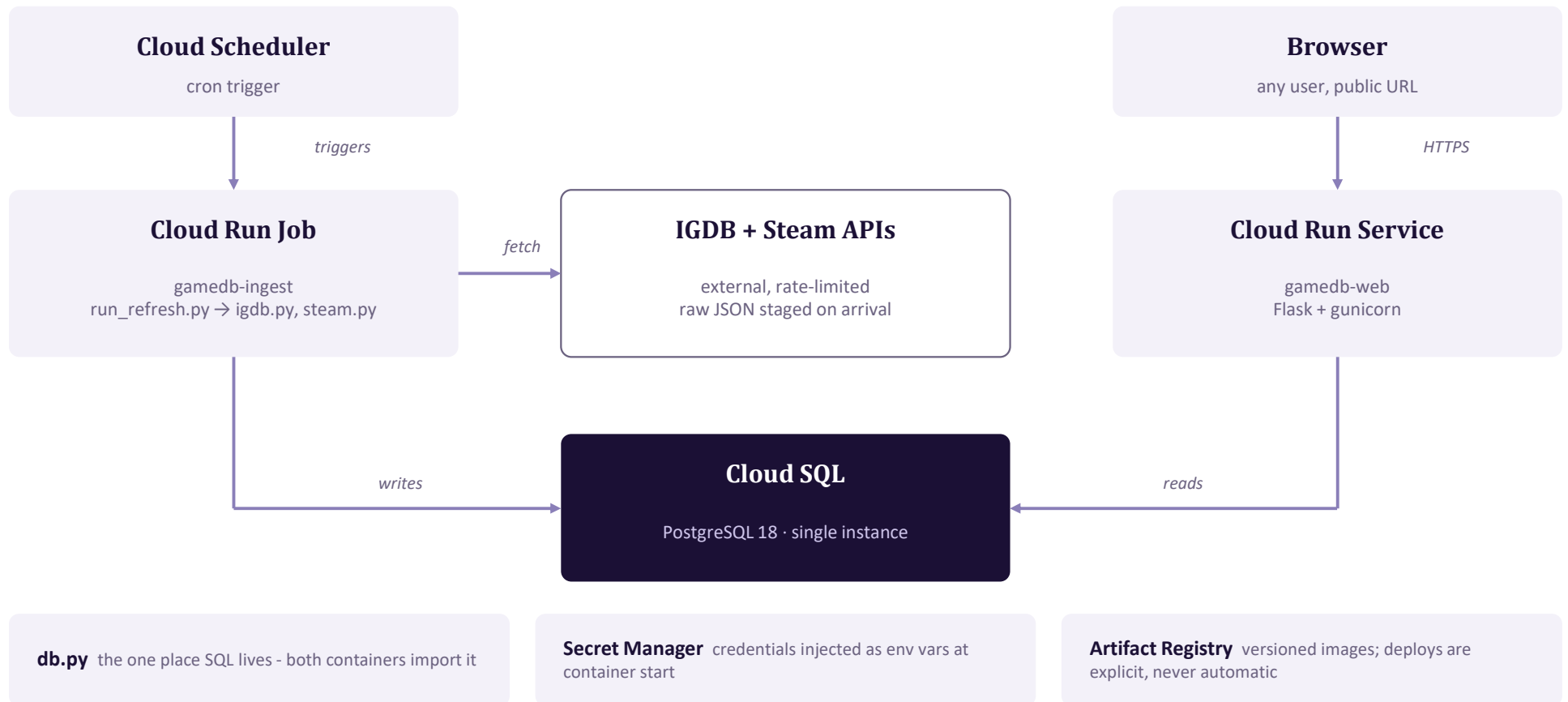
Holds all four credentials; injected into containers as env vars at start.

Supporting GCP: Cloud SQL Auth Proxy (local dev) · Cloud Logging (job debugging) · IAM service accounts · Cloud Shell

Application stack: Python 3.12 · Flask · gunicorn · psycopg 3 · Jinja2 · Docker · IGDB + Steam Web APIs

ZOOM IN · TWO CONTAINERS, ONE DATABASE

Architecture: The Moving Parts



Database: Cloud SQL for PostgreSQL

~305K

games in the catalog

~5.4M

rows across 10 tables

~3.3M

game-attribute links

● Why relational

- The data is heavily relational: games ↔ scores ↔ sources ↔ attributes, with joins everywhere in the ranking query
- Rejected Datastore/Firestore - a document store makes the ranking aggregation miserable
- Rejected Spanner and AlloyDB - horizontal scale we will never need at 5M rows

● Schema shape

- One canonical game row, separate from each source's record of it
- Scores share one normalized shape, so a new source is new rows, not new columns
- Attributes are generic tags - a new kind (theme, keyword) needs no schema change
- raw_fetch stages every API response as JSONB before reconciliation

How we use it: all SQL lives in db.py · ranking is one multi-CTE query · every write is an idempotent upsert · separate read-only and read-write database users

Lessons Learned

1 Containers forget everything

Our ingestion script cached progress to a local file and resumed from it - perfectly, on a laptop. Each Cloud Run task gets a fresh filesystem, so a retry restarted from zero. Cost us ~12 hours of duplicate work. State belongs in the database.

2 Read the defaults before the docs

Cloud Run Jobs default to a 10-minute task timeout. Our catalog walk takes 8 hours. A too-short timeout killed one run at 88% complete.

3 Test at the smallest possible loop

Four failed job executions chasing a credential bug, minutes of log-reading each. The fix that worked: request the token locally with the exact bytes from Secret Manager, before spending a container start on it.

4 Postgres privileges are narrower than they look

GRANT ... ON SCHEMA applies only to the database you are connected to, and ON ALL TABLES covers only tables that exist right now. Both print success either way.

5 Secrets leak through error messages

Credentials passed as URL parameters end up in the exception text, and the exception ends up in Cloud Logging. Put them in the request body instead.

6 The best managed service is often the one you skip

Managed Redis was the obvious answer to a slow query - but an index and a materialized view fix the cause, with no VPC connector and no meter running.